1. What is the role of an Error Boundary in React?

An **Error Boundary** is a special React component that is designed to catch JavaScript errors that occur anywhere in the component tree below it. It prevents the entire application from crashing and allows developers to display a fallback UI (like an error message or a blank screen).

Key Points:

- Error Boundaries catch errors during rendering, in lifecycle methods, and in constructors of thewhole tree below them.
- They do not catch errors inside event handlers, asynchronous code (like setTimeout), orserver-side rendering.
- A typical Error Boundary uses the componentDidCatch() and getDerivedStateFromError()methods.

Example:

```
class ErrorBoundary extends
React.Component {{   constructor(props) {{
super(props);

this.state = {{ hasError: false }};

   }}

  static getDerivedStateFromError(error) {{
return {{ hasError: true }};

  }}

  componentDidCatch(error, info) {{
console.log(error, info);   }}

  render() {{
 if (this.state.hasError) {{
return <h2>Something went wrong.</h2>;
}}
return this.props.children;   }}
}}
```

2. How does the try...catch block differ from an Error Boundary in handling errors in React?

try...catch is a JavaScript feature that handles errors in **imperative code** (like functions or event handlers). It cannot catch errors that occur during React's rendering phase.

Key Differences:

- **Error Boundary** → Catches rendering and lifecycle errors inside components.
- **try...catch** → Handles errors in normal JavaScript logic or event handlers.
- **Error Boundary** works at the component level; **try...catch** works at the code block level.
- **Error Boundary** provides a fallback UI; **try...catch** does not.

Example of try...catch:

```
function handleClick() {{   try {{     throw
new Error("Something went wrong");   }}
catch (error) {{
console.error(error.message);   }}
}}
```